

UNIVERSIDADE ESTADUAL PAULISTA "JÚLIO
DE MESQUITA FILHO"
FACULDADE DE CIÊNCIA E TECNOLOGIA (FCT)

ANÁLISE APROFUNDADA DE MÉTODOS DE
OTIMIZAÇÃO LINEAR:

DO SIMPLEX PADRÃO À DECOMPOSIÇÃO EM LARGA ESCALA

Relatório técnico apresentado à disciplina de Pesquisa Operacional, como requisito parcial para avaliação.

Presidente Prudente

2025

Sumário

1	Introdução e Contextualização Histórica	3
2	Fundamentos Matemáticos e o Simplex Padrão	3
2.1	A Forma Padrão e a Geometria	3
2.2	Partição Básica e Soluções Viáveis	4
2.3	O Mecanismo do Simplex via Tableau (Padrão)	4
2.3.1	Análise de Complexidade do Simplex Padrão	5
3	O Método Simplex Revisado: Uma Abordagem Eficiente	5
3.1	A Filosofia da Computação Sob Demanda	5
3.2	O Algoritmo Matemático Revisado Detalhado	6
3.2.1	Passo 1: Cálculo dos Multiplicadores Simplex (Vetor Dual)	6
3.2.2	Passo 2: Pricing (Seleção da Variável de Entrada)	6
3.2.3	Passo 3: Cálculo da Coluna Atualizada	6
3.2.4	Passo 4: Teste da Razão (Variável de Saída)	7
3.2.5	Passo 5: Atualização da Inversa da Base	7
3.3	Vantagens Computacionais Críticas	7
3.3.1	Preservação da Esparsidade	7
3.3.2	Estabilidade Numérica	7
3.3.3	Comparativo de Operações	8
4	Exemplo Prático de Aplicação do Simplex Revisado	8
5	Aplicações Práticas: Quando usar o Revisado?	9
5.1	Problemas de Escala de Tripulação (Crew Scheduling)	9
5.2	Problemas Dinâmicos e Estocásticos	10
6	Princípio de Decomposição de Dantzig-Wolfe	10
6.1	Estrutura Bloco-Angular	10
6.2	Reformulação do Problema Mestre e Convexidade	10

6.3	O Ciclo de Geração de Colunas	11
7	Conclusão	11
8	Referências Bibliográficas	11

1 Introdução e Contextualização Histórica

A Programação Linear (PL) representa um dos maiores triunfos da matemática aplicada do século XX. Desde a formulação do problema da dieta por George Stigler até o desenvolvimento do algoritmo Simplex por George Dantzig em 1947, a capacidade de otimizar uma função objetivo linear sujeita a restrições lineares revolucionou a economia, a logística e a engenharia.

No entanto, a complexidade dos problemas do mundo real cresceu exponencialmente nas últimas décadas. O que antes eram problemas manuais de planejamento de produção com dezenas de variáveis, hoje são problemas de otimização de redes energéticas ou escalas de tripulação aérea envolvendo milhões de variáveis e centenas de milhares de restrições.

Neste cenário, o Simplex Padrão ("Tableau Simplex"), embora didaticamente excelente, mostra-se inadequado para computação em larga escala. Este relatório explora as razões matemáticas para essa inadequação e apresenta as soluções robustas desenvolvidas pela comunidade científica: o Simplex Revisado e a Decomposição de Dantzig-Wolfe.

2 Fundamentos Matemáticos e o Simplex Padrão

Para compreender as vantagens do método revisado, é imperativo primeiro formalizar a álgebra do método padrão e identificar onde ocorrem seus gargalos computacionais.

2.1 A Forma Padrão e a Geometria

Considere o problema de Programação Linear na sua forma padrão:

$$\begin{aligned} \text{Maximizar} \quad & z = c^T x \\ \text{Sujeito a} \quad & Ax = b \\ & x \geq 0 \end{aligned} \tag{1}$$

Onde:

- A é uma matriz de restrições de dimensão $m \times n$ (com $n > m$ e posto completo m).
- $c \in \mathbb{R}^n$ é o vetor de custos.
- $b \in \mathbb{R}^m$ é o vetor de recursos (lado direito).
- $x \in \mathbb{R}^n$ é o vetor de variáveis de decisão.

O conjunto de soluções viáveis $P = \{x | Ax = b, x \geq 0\}$ forma um poliedro convexo em $\mathbb{R}^n$. O Teorema Fundamental da Programação Linear garante que, se existe uma solução ótima finita, ela reside em um dos vértices (pontos extremos) deste poliedro.

2.2 Partição Básica e Soluções Viáveis

A estratégia algébrica do Simplex baseia-se na partição da matriz A . Escolhemos m colunas linearmente independentes para formar uma base B . As demais $n - m$ colunas formam a matriz não-básica N . Analogamente, particionamos as variáveis x em (x_B, x_N) e os custos c em (c_B, c_N) .

A equação de restrições $Ax = b$ pode ser reescrita como:

$$Bx_B + Nx_N = b \quad (2)$$

Pré-multiplicando por B^{-1} , isolamos as variáveis básicas:

$$x_B = B^{-1}b - B^{-1}Nx_N \quad (3)$$

Uma Solução Básica Viável (SBV) ocorre quando fixamos $x_N = 0$. Logo, $x_B = B^{-1}b$. Se $x_B \geq 0$, estamos em um vértice do poliedro viável.

A função objetivo pode ser reescrita substituindo x_B :

$$z = c_B^T x_B + c_N^T x_N \quad (4)$$

$$z = c_B^T (B^{-1}b - B^{-1}Nx_N) + c_N^T x_N \quad (5)$$

$$z = c_B^T B^{-1}b + (c_N^T - c_B^T B^{-1}N)x_N \quad (6)$$

O termo entre parênteses é o vetor de **Custos Reduzidos**, denotado por $\hat{c}_N^T$.

2.3 O Mecanismo do Simplex via Tableau (Padrão)

No método Simplex Padrão, mantemos e atualizamos iterativamente uma tabela (tableau) que representa explicitamente o sistema de equações lineares transformado pela base atual. Um tableau típico contém explicitamente a matriz $B^{-1}A$.

A cada iteração, realizamos operações de linha (pivoteamento de Gauss-Jordan) para transformar uma coluna não-básica em um vetor unitário da base canônica.

2.3.1 Análise de Complexidade do Simplex Padrão

Aqui reside a ineficiência crítica. Suponha um problema onde $m = 1.000$ (restrições) e $n = 10.000$ (variáveis).

1. **Atualização Desnecessária:** Ao realizar o pivoteamento, o Simplex Padrão atualiza todas as n colunas da matriz. Ou seja, calculamos $B^{-1}a_j$ para todas as variáveis não-básicas j , mesmo aquelas que têm custo reduzido muito negativo e provavelmente nunca entrarão na base.
2. **Custo Computacional:** Uma operação de pivô requer aproximadamente $m \times n$ multiplicações e adições. Em problemas de grande porte, esse custo torna-se proibitivo.
3. **Preenchimento (Fill-in):** Em problemas reais, a matriz A é frequentemente esparsa (contém muitos zeros). Ao multiplicar $B^{-1}A$ explicitamente e fazer operações de linha, os zeros originais tendem a se tornar números não-nulos (fill-in). Isso aumenta drasticamente o uso de memória e o esforço computacional.

3 O Método Simplex Revisado: Uma Abordagem Eficiente

O Método Simplex Revisado altera fundamentalmente a forma como os cálculos são realizados, mantendo a mesma trajetória geométrica (sequência de vértices) do método padrão.

3.1 A Filosofia da Computação Sob Demanda

A premissa do método revisado é: não precisamos ver o tableau inteiro para tomar a decisão de qual vértice visitar a seguir. Precisamos apenas de:

- Os custos reduzidos ($\hat{c}$) para decidir quem entra na base.
- A coluna atualizada da variável de entrada (α) para decidir quem sai (razão mínima).

Para isso, em vez de transformar a matriz A inteira (uma abordagem de "Força Bruta"), trabalhamos sempre com a matriz A original e mantemos apenas a inversa da base B^{-1} atualizada (uma abordagem "Cirúrgica").

3.2 O Algoritmo Matemático Revisado Detalhado

Vamos detalhar o passo a passo matemático, enfatizando as diferenças de cálculo em relação ao padrão.

3.2.1 Passo 1: Cálculo dos Multiplicadores Simplex (Vetor Dual)

Em vez de calcular o custo reduzido diretamente, primeiro calculamos o vetor π (multiplicadores simplex ou variáveis duais):

$$\pi^T = c_B^T B^{-1} \quad (7)$$

Isso equivale a resolver o sistema linear $B^T \pi = c_B$.

3.2.2 Passo 2: Pricing (Seleção da Variável de Entrada)

Para encontrar a variável de entrada, calculamos:

$$\hat{c}_j = c_j - \pi^T a_j \quad \forall j \in \text{Não-Básicas} \quad (8)$$

Note a eficiência: este cálculo é um produto interno entre vetores. Não precisamos calcular a coluna inteira atualizada. Além disso, podemos usar estratégias de *Pricing Parcial*, calculando $\hat{c}_j$ apenas para um subconjunto de variáveis até encontrar uma candidata positiva, economizando CPU.

Se $\hat{c}_j \leq 0$ para todo j , PARE. A solução atual é ótima. Caso contrário, selecione k tal que $\hat{c}_k > 0$.

3.2.3 Passo 3: Cálculo da Coluna Atualizada

Somente agora, e apenas para a variável vencedora x_k , calculamos sua representação na base atual:

$$\alpha = B^{-1} a_k \quad (9)$$

Isso equivale a resolver o sistema linear $B\alpha = a_k$.

3.2.4 Passo 4: Teste da Razão (Variável de Saída)

Idêntico ao método padrão. Calculamos o vetor de solução atual $\bar{b} = B^{-1}b$ (normalmente já disponível da iteração anterior) e fazemos:

$$\theta = \min \left\{ \frac{\bar{b}_i}{\alpha_i} : \alpha_i > 0 \right\} \quad (10)$$

Seja x_{B_r} a variável básica associada ao índice r que minimiza a razão. Ela deixará a base.

3.2.5 Passo 5: Atualização da Inversa da Base

Este é o coração do método revisado. Temos uma nova base B_{nova} que difere de B_{velha} por apenas uma coluna. Não precisamos inverter a matriz do zero (custo $O(m^3)$). Usamos matrizes elementares. A nova inversa é dada por:

$$B_{nova}^{-1} = E \cdot B_{velha}^{-1} \quad (11)$$

Onde E é uma matriz elementar (Matriz Eta) que transforma a coluna r -ésima da identidade na coluna pivô.

3.3 Vantagens Computacionais Críticas

3.3.1 Preservação da Esparsidade

Como a matriz A nunca é alterada, seus zeros permanecem zeros. Isso economiza imensa quantidade de memória. Apenas a matriz B^{-1} (ou sua representação fatorada) precisa ser armazenada.

3.3.2 Estabilidade Numérica

No Simplex Padrão, erros de arredondamento se propagam cumulativamente por todas as mn entradas do tableau a cada iteração. Após milhares de iterações, o tableau pode se tornar "lixo numérico".

No Revisado, os erros se acumulam apenas na inversa da base. Periodicamente, podemos descartar a inversa acumulada e recalcular B^{-1} do zero usando decomposição LU de alta precisão sobre a matriz original. Isso "reseta" o erro numérico, uma funcionalidade impossível no método padrão.

3.3.3 Comparativo de Operações

- **Padrão:** Custo de $O(m \cdot n)$ por iteração.
- **Revisado:** Custo de $O(m^2)$ por iteração (assumindo cálculo denso).

Conclusão: Quando $n \gg m$ (muito mais variáveis que restrições), o Revisado é ordens de grandeza mais rápido.

4 Exemplo Prático de Aplicação do Simplex Revisado

Para consolidar a teoria apresentada, aplicaremos o algoritmo revisado passo a passo em um problema numérico.

Problema: Maximizar $Z = 4x_1 + 3x_2$

$$\text{Sujeito a: } \begin{cases} 2x_1 + x_2 \leq 10 \\ x_1 + x_2 \leq 8 \\ x_1, x_2 \geq 0 \end{cases}$$

Preparação (Forma Padrão): Introduzimos as variáveis de folga x_3 e x_4 .

$$\text{Max } Z = 4x_1 + 3x_2 + 0x_3 + 0x_4$$

$$\begin{cases} 2x_1 + 1x_2 + 1x_3 + 0x_4 = 10 \\ 1x_1 + 1x_2 + 0x_3 + 1x_4 = 8 \end{cases}$$

A matriz A , o vetor b e o vetor c são:

$$A = \begin{bmatrix} 2 & 1 & 1 & 0 \\ 1 & 1 & 0 & 1 \end{bmatrix}, \quad b = \begin{bmatrix} 10 \\ 8 \end{bmatrix}, \quad c = \begin{bmatrix} 4 & 3 & 0 & 0 \end{bmatrix}^T$$

Iteração 1:

1. **Base Inicial:** Escolhemos as folgas. $B = I = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$, $B^{-1} = I$. Variáveis básicas $x_B = \{x_3, x_4\}$. Custos básicos $c_B = [0, 0]$.

2. **Passo 1 (Multiplicadores π):**

$$\pi^T = c_B^T B^{-1} = [0, 0] \cdot I = [0, 0]$$

3. **Passo 2 (Pricing):** Calculamos $\hat{c}_j = c_j - \pi^T a_j$.

$$\hat{c}_1 = 4 - [0, 0] \cdot \begin{pmatrix} 2 \\ 1 \end{pmatrix} = 4$$

$$\hat{c}_2 = 3 - [0, 0] \cdot \begin{pmatrix} 1 \\ 1 \end{pmatrix} = 3$$

O maior custo reduzido positivo é $\hat{c}_1 = 4$. Logo, x_1 **entra na base**.

4. **Passo 3 (Coluna Atualizada):**

$$\alpha = B^{-1}a_1 = I \cdot \begin{pmatrix} 2 \\ 1 \end{pmatrix} = \begin{pmatrix} 2 \\ 1 \end{pmatrix}$$

5. **Passo 4 (Teste da Razão):** Solução atual $\bar{b} = B^{-1}b = [10, 8]^T$.

$$\text{Razão 1 } (x_3): 10/2 = 5$$

$$\text{Razão 2 } (x_4): 8/1 = 8$$

A menor razão é 5. Logo, a variável básica da primeira linha (x_3) **sai da base**.

6. **Resultado da Iteração 1:** Nova base será $\{x_1, x_4\}$.

Este exemplo demonstra claramente que não foi necessário calcular a coluna atualizada de x_2 ou atualizar toda a matriz de restrições, economizando operações.

5 Aplicações Práticas: Quando usar o Revisado?

O Simplex Revisado é o padrão da indústria em solvers comerciais como CPLEX, Gurobi e XPRESS. Ele brilha em cenários específicos:

5.1 Problemas de Escala de Tripulação (Crew Scheduling)

Companhias aéreas precisam alocar milhares de tripulantes para milhares de voos.

- **Estrutura:** Milhões de variáveis (possíveis combinações de rotas/escalas) e "apenas" alguns milhares de restrições (cobrir todos os voos, leis trabalhistas).
- **Por que o Revisado?** Com $n \approx 1.000.000$ e $m \approx 5.000$, manter um tableau de $5 \cdot 10^9$ entradas é impossível. O Revisado calcula custos reduzidos sob demanda, permitindo a resolução eficiente.

5.2 Problemas Dinâmicos e Estocásticos

Em problemas de múltiplos estágios onde novas variáveis são geradas dinamicamente (Geração de Colunas), o método revisado é obrigatório, pois a matriz A completa nem sequer existe no início do processo.

6 Princípio de Decomposição de Dantzig-Wolfe

A evolução natural para resolver problemas que excedem até mesmo a capacidade do Simplex Revisado é a decomposição. O método de Dantzig-Wolfe aborda problemas com estrutura **Bloco-Angular**.

6.1 Estrutura Bloco-Angular

Imagine uma corporação com duas fábricas. Cada fábrica tem suas restrições internas (máquinas, turnos), mas ambas competem por um orçamento corporativo global. A matriz de restrições tem a forma:

$$A = \begin{bmatrix} L_1 & L_2 & \dots & L_K \\ A_1 & 0 & \dots & 0 \\ 0 & A_2 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & A_K \end{bmatrix} \quad (12)$$

Onde L_k são as restrições de ligação (globais) e A_k são os blocos independentes.

6.2 Reformulação do Problema Mestre e Convexidade

A ideia central baseia-se na **Teoria da Convexidade** e no **Teorema da Representação de Poliedros**. Qualquer solução viável para o bloco k (x_k) pode ser expressa como uma combinação convexa dos vértices do poliedro deste bloco.

Seja $V_k = \{v_{k1}, v_{k2}, \dots, v_{kp}\}$ o conjunto de vértices do poliedro definido por $A_k x_k = b_k$. Então:

$$x_k = \sum_j \lambda_{kj} v_{kj}, \quad \sum \lambda_{kj} = 1, \quad \lambda \geq 0 \quad (13)$$

Substituímos x_k no problema original pelas variáveis λ . O problema original nas variáveis x (que tem muitas restrições) transforma-se em um **Problema Mestre** nas variáveis λ (que tem poucas restrições globais, mas muitas variáveis — uma para cada vértice de cada subproblema).

6.3 O Ciclo de Geração de Colunas

Não podemos listar todos os vértices v_{kj} a priori, pois são muitos. Usamos o Simplex Revisado no Problema Mestre e geramos as colunas (os vértices) sob demanda.

1. **Mestre Restrito:** Resolvemos o problema mestre com apenas alguns vértices conhecidos. Obtemos os preços sombra duais (π) das restrições de ligação.
2. **Subproblemas:** Enviamos π para as divisões. Cada divisão k resolve um subproblema local tentando minimizar seu custo reduzido ajustado pelo preço sombra:

$$\min(c_k - \pi^T L_k)x_k \quad \text{s.a.} \quad A_k x_k = b_k$$

3. **Integração:** A solução ótima do subproblema é um vértice v_{kj}^* . Se ele tiver custo reduzido negativo no mestre, adicionamos essa nova coluna ao Mestre e reotimizamos.

Este processo imita um mercado econômico: o centro (Mestre) define preços para recursos escassos, e as filiais (Subproblemas) propõem planos de produção baseados nesses preços.

7 Conclusão

A transição do Simplex Padrão para o Revisado não é apenas uma mudança de implementação, mas uma mudança paradigmática de "processamento em lote" para "processamento sob demanda". O Simplex Padrão é pedagogicamente inestimável para entender a geometria dos vértices, mas computacionalmente inviável para o mundo real devido à densidade e complexidade $O(mn)$.

O Simplex Revisado, ao explorar a álgebra matricial esparsa e focar nas operações essenciais (B^{-1}), viabiliza a resolução de problemas industriais. Para além disso, o Princípio de Decomposição de Dantzig-Wolfe eleva essa capacidade, permitindo que sistemas massivos e descentralizados sejam otimizados de forma coordenada, pavimentando o caminho para a moderna otimização em larga escala.

8 Referências Bibliográficas

1. ARENALES, M.; ARMENTANO, V.; MORABITO, R.; YANASSE, H. *Pesquisa Operacional: Para Cursos de Engenharia*. Rio de Janeiro: Elsevier, 2007.

2. BAZARAA, M. S.; JARVIS, J. J.; SHERALI, H. D. *Linear Programming and Network Flows*. 4. ed. New Jersey: John Wiley & Sons, 2010.
3. BERTSEKAS, D. P. *Linear Network Optimization: Algorithms and Codes*. MIT Press, 1991.
4. CHVÁTAL, V. *Linear Programming*. New York: W. H. Freeman, 1983.
5. DANTZIG, G. B. *Linear Programming and Extensions*. Princeton University Press, 1963.
6. LASDON, L. S. *Optimization Theory for Large Systems*. New York: Macmillan, 1970.
7. VANDERBEI, R. J. *Linear Programming: Foundations and Extensions*. 4. ed. Springer, 2014.